Predicting Asset Returns via a Structured Strategy Space: A Two-Layer Mathematical Framework for Adaptive Trading

Final Project Report

Joon Jeremy Chun

April 2026

Abstract

This report presents a two-layer mathematical framework for predicting asset returns using a structured strategy space. The first layer applies a library of domain-grounded trading strategies—each parameterized and evaluated over historical price data—to construct a design matrix $A \in \mathbb{R}^{T \times N}$ whose columns are strategy signal vectors. The second layer fits a sparse linear model $\hat{y} = A\boldsymbol{\beta}$ on a rolling one-year window to predict the 130-day forward return and derive a continuous portfolio weight. Three stages of increasing complexity are evaluated: Stage A (single best strategy via grid search), Stage B (constrained ensemble), and Stage C (ML-guided sparse prediction). Applied to the SPDR Gold ETF (GLD) over a five-year forward test (2020–2024), Stage C achieves a capital efficiency Sharpe ratio of 1.12 (top-20 basis, leakage-corrected), outperforming Stage A and B in every evaluation year. A key finding is that prediction horizon alignment—matching the model's target horizon to the evaluation window used in Layer 1—is critical for out-of-sample generalization.

1 Introduction

A fundamental challenge in quantitative finance is constructing predictive models that generalize across market regimes without overfitting to short historical windows. Standard machine learning approaches face two compounding difficulties in this domain: the number of candidate features grows rapidly with the breadth of the strategy search, yet the effective sample size is constrained by regime nonstationarity—extending the training window introduces data from market environments that may be structurally different from the current one.

This project proposes a solution grounded in a mathematical observation: every parameterized trading strategy applied to a price series of length T produces a discrete signal vector $\boldsymbol{s}_\theta(t) \in \{-1, 0, +1\}^T$, which is a point in $\mathbb{R}^T$. The collection of all such vectors, indexed by strategy family and parameter configuration, forms a *strategy vector space* $\mathcal{S}$. Rather than treating strategy outputs as black-box features, this framework treats them as basis functions: the top-N performing strategies per family are assembled into a design matrix A, and sparse linear regression is used to learn a prediction in the subspace spanned by A.

This decomposition separates the problem into two layers with complementary properties. Layer 1 is fixed and encodes domain knowledge: it converts raw price data into structured signals using interpretable trading rules. Layer 2 is adaptive and data-driven: it learns which linear combination of signals best predicts future returns on a rolling basis. The result is a prediction system that can adapt to regime changes while remaining well-conditioned on a one-year training window.

The system is evaluated through a rolling forward test spanning 2020–2024 on GLD and deployed in a live paper-trading environment on a Raspberry Pi, generating daily signals and executing fractional-share tranche orders via the Alpaca API.

2 Background

2.1 Combining Alpha Signals in Quantitative Finance

A central problem in quantitative finance is combining multiple trading signals—each capturing a different aspect of price dynamics—into a single portfolio decision. Kakushadze [2015] formalize this as a bounded regression problem: given a set of alpha streams, the optimal combination weights are found by solving a constrained least-squares problem over a historical evaluation window. Their framework motivates the design of Layer 1 in the present work, where the top-N strategies per family serve as individual alpha streams and Layer 2 learns to combine them via regularized regression rather than constrained optimization. The key distinction is that the present framework does not fix the combination weights—it re-estimates them monthly as market conditions evolve, and uses the combined signal to *predict* future returns rather than maximize past returns.

2.2 Sparse Linear Regression

The Lasso [Tibshirani, 1996] minimizes the residual sum of squares subject to an ℓ_1 constraint on the coefficient vector, which drives many coefficients to exactly zero and thus performs implicit feature selection. This property is essential in the present setting: with up to 80 strategy signals as candidate predictors and only one year of daily training data (≈ 260 observations), an unconstrained linear model would severely overfit. Ridge regression adds an ℓ_2 penalty that shrinks all coefficients without eliminating any, and the Elastic Net combines both penalties to handle correlated predictors. All three regularization methods are used in Layer 2, with the active model selected monthly by cross-validated mean squared error.

3 Methods

3.1 Layer 1: The Strategy Vector Space

For a price series $p(t)$, $t = 1, \ldots, T$, a trading strategy with parameter vector θ produces a signal time series $\boldsymbol{s}_\theta = (s_\theta(1), \ldots, s_\theta(T))^\top \in \mathbb{R}^T$. Four strategy families are used:

1. **Adaptive Band**: A moving average of length w is computed; the signal is $+1$ when the price exceeds the moving average by k_u standard deviations and -1 when it falls below by $|k_l|$ standard deviations. Parameters: $w \in \{5, 10, 22, 65, 130, 260\}$, $k_u, k_l \in [-3.0, 3.0]$ at step 0.1.

2. **MA Crossover**: A $+1$ signal is generated when a short moving average of length s crosses above a long moving average of length l, and -1 when it crosses below. Parameters: $s \in [5, 95]$, $l \in [s{+}5, 140]$ at step 5.

3. **Adaptive Volatility Band**: Similar to Adaptive Band but the band width is defined in terms of rolling volatility rather than fixed standard deviations of the price level.

4. **Fear-Greed Candle-Volume**: A composite signal based on candlestick body size relative to σ and trading volume relative to the rolling average volume.

Grid search over all parameter configurations is parallelized using `joblib` (4.4× speedup on a 20-core workstation). The top-N candidates per family, ranked by total return in the one-year selection window, are selected to form the columns of the strategy matrix $A \in \mathbb{R}^{T \times N_{\text{total}}}$, where $N_{\text{total}} = 4N$. In the primary configuration, $N = 10$, giving $A \in \mathbb{R}^{T \times 40}$.

3.2 Layer 2: Sparse Linear Prediction

Let $\boldsymbol{y} \in \mathbb{R}^T$ be the vector of 130-day forward returns, and let A be the strategy matrix constructed from the selection window. Layer 2 minimizes the regularized objective:

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \left\{ \frac{1}{T} \|A\boldsymbol{\beta} - \boldsymbol{y}\|_2^2 + \lambda_1 \|\boldsymbol{\beta}\|_1 + \frac{\lambda_2}{2} \|\boldsymbol{\beta}\|_2^2 \right\}, \tag{1}$$

where $\lambda_1 = 0$ for Ridge, $\lambda_2 = 0$ for Lasso, and both $\lambda_1, \lambda_2 > 0$ for Elastic Net. Hyperparameters are selected via 5-fold cross-validation with a time-aware split. The active model on each anchor date is the one minimizing cross-validated MSE. The predicted return $\hat{y}_t = A_t\hat{\boldsymbol{\beta}}$ is scaled by the 95th quantile of in-sample predictions to produce a portfolio weight $w_t \in [0, 1]$.

3.3 Three-Stage Evaluation Framework

Three progressively complex stages are defined to isolate the contribution of each modeling decision:

- **Stage A**: At each anchor date, the single highest-ranked strategy from the grid search is selected and deployed for the hold period. No combination or prediction is performed.
- **Stage B**: All top-N strategies per family are combined with weights determined by constrained optimization over the selection window. The weight vector is fixed for the hold period.
- **Stage C**: The sparse linear model (Layer 2) is fit to predict the 130-day forward return from the strategy signals in A. The predicted return is converted to a continuous portfolio weight w_t updated daily.

3.4 Rolling Forward Validation

The evaluation protocol uses 79 anchor dates from December 2019 to March 2026 at monthly intervals. At each anchor date: (1) strategy parameters are optimized on the preceding one-year selection window; (2) the Layer 2 model is fit on the same window; (3) the signal is deployed for 130 trading days (Stage C) or until the next anchor date (Stages A and B). The forward evaluation period is 2020–2024, comprising five complete calendar years.

To approximate realistic capital deployment, a rolling tranche structure is used: the total capital is divided into 130 equal tranches, one of which matures and is reinvested each trading day. This ensures capital is continuously deployed and that reported returns reflect realistic, regime-diverse performance rather than cherry-picked entry points.

Performance is reported as (i) raw strategy return, (ii) average capital exposure, and (iii) capital efficiency, defined as return divided by average exposure—the return earned per dollar deployed.

4 Results

4.1 Stage Comparison: A, B, and C

Table 1 reports per-year and Sharpe ratio results for all three stages applied to GLD over 2020–2024.

Stage C achieves the highest return in every year, with a capital efficiency Sharpe of 1.08. Critically, in the 2021 bear year (GLD −6.2%), Stage C returns +0.8% by reducing exposure to 29%—demonstrating that the model has learned not only which strategies to weight but when to withhold

Table 1: GLD forward-test results by stage (hold 130d, 1M update interval). Ret = strategy return; Exp = average capital exposure; efficiency = Ret/Exp.

Year	B&H	**A**: Single	**B**: Ensemble	**C**: ML-Guided	
		Ret / Exp	Ret / Exp	Ret	Exp
2020	+23.9%	+4.4% / 27%	+6.4% / 32%	+10.9%	57%
2021	−6.2%	+0.3% / 25%	+0.3% / 25%	+0.8%	29%
2022	+0.8%	−0.9% / 46%	−2.3% / 42%	−0.2%	31%
2023	+11.8%	+1.6% / 25%	+2.0% / 23%	+4.0%	32%
2024	+27.0%	+5.9% / 21%	+8.0% / 29%	+16.4%	65%
Sharpe (eff.)	—	≈0.5	≈0.7	**1.08**	

capital. The progression from A to B to C in 2024 (+5.9% → +8.0% → +16.4%) illustrates the incremental value of each modeling layer.

4.2 Prediction Horizon Analysis

The prediction horizon—the number of days into the future that Layer 2 is trained to predict—is a critical hyperparameter. To identify the optimal horizon, all integer multiples of five from 5 to 260 days were evaluated, and the model was run with matching training and hold periods for each.

The 130-day horizon was most consistently selected across the evaluation period. A comparison between the primary model (h130, train and hold 130 days) and a short-horizon alternative (h45, train and hold 45 days) reveals a striking pattern: while h45 matches or exceeds h130 in bull years (2024: +29% vs +25% capital efficiency), it catastrophically underperforms in the 2021 bear year (−18% vs +3%). This asymmetry is attributed to *feature–target misalignment*: the columns of A are constructed using a six-month (≈130-day) evaluation window in Layer 1, so the features are calibrated to exactly the scale that the h130 model is predicting. The h45 model, by contrast, attempts to predict a shorter horizon using features that encode six-month market structure, introducing a systematic mismatch that amplifies noise in adversarial market conditions (Figure 4).

4.3 Top-N Feature Expansion

The number of strategies included per family (N) controls the dimensionality of the design matrix A. A systematic scan over $N \in \{10, 20, 30, \ldots, 50\}$ was conducted using a training protocol that strictly prevents label leakage: training rows whose target end date extends past the anchor date are excluded, and feature normalization statistics are anchored to the anchor date in all evaluation windows.

Under this corrected protocol, $N = 20$ achieves the highest capital efficiency Sharpe of **1.12**, outperforming both $N = 10$ (Sharpe 0.94) and $N = 30$ (Sharpe 1.07). The initial drop at $N = 20$ versus $N = 10$ in the uncorrected results was attributable to look-ahead leakage in the training labels, which inflated the apparent advantage of the smaller basis. After correction, the additional

10 strategies per family provide meaningful complementary signal coverage without introducing excessive noise. For the short-horizon alternative (h45), Sharpe remains consistently low across all N (0.17–0.49), confirming that the horizon alignment is the dominant driver of performance rather than feature count (Figure 5).

4.4 Live Deployment

The full pipeline is deployed on a Raspberry Pi 4 running a systemd timer that triggers daily at 13:00 PDT on trading days. At each trigger: (1) the latest GLD price is fetched via the Alpaca Markets API; (2) Layer 2 generates a portfolio weight using the most recent anchor model; (3) a fractional-share tranche order is submitted to Alpaca paper trading; and (4) a signal report is emailed automatically. As of April 2026, the active signal is BUY with weight 1.00 under a Ridge model (anchor 2026-03-31), predicting +55.6% over the next 130 days. The signal and order log are committed to version control daily.

5 Discussion and Conclusions

5.1 Why the Two-Layer Architecture Works

The central contribution of this framework is the explicit separation between nonlinear feature extraction (Layer 1) and linear prediction (Layer 2). Layer 1 performs a fixed, domain-grounded transformation of raw price data into structured signals using interpretable trading rules. This transformation is not learned from data, which means it does not consume sample budget. Layer 2 then operates in a space of N_{total} dimensions rather than on raw prices, where the columns of A are orthogonalized by strategy selection and their meanings are interpretable.

This structure also explains the horizon alignment finding. Layer 1 selects the top-N strategies by total return over a six-month evaluation window; those strategies are therefore best at capturing six-month price dynamics. When Layer 2 is trained to predict a six-month return using those strategies as features, the features are perfectly calibrated to the prediction target. Shorter-horizon models lack this alignment, explaining their fragility.

5.2 Capital Efficiency as the Primary Metric

Standard return comparisons are misleading for partial-exposure strategies: a system that earns +10% while fully invested is less impressive than one that earns +10% while holding only 30% of capital. By measuring return per dollar deployed, the capital efficiency metric places all strategies and horizons on a common footing. Stage C dominates on this metric despite having lower absolute returns than B&H in most years, because it concentrates capital deployment in high-confidence periods.

5.3 Limitations and Future Directions

Several limitations should be acknowledged. First, the framework is evaluated on a single asset over a five-year period; out-of-sample generalization to other markets or time periods is not guaranteed. Second, the strategy families are manually specified; a more general framework would learn the structure of Layer 1 from data. Third, transaction costs and market impact are not modeled, which would likely reduce absolute returns.

Future work will extend the framework to a multi-asset allocation setting, where idle capital from one asset can be redeployed into another based on uncorrelated signals. Additional planned extensions include widening the Layer 1 evaluation horizons to 18 and 24 months, and transitioning the live system from paper trading to live execution.

5.4 Conclusions

This work demonstrates that a two-layer mathematical architecture combining interpretable trading signals (Layer 1) with sparse linear prediction (Layer 2) can produce a well-behaved, adaptable return predictor using only one year of training data. The key design principles—strategy vector space construction, rolling validation, and prediction horizon alignment—each contribute measurably to out-of-sample performance. Stage C achieves a five-year capital efficiency Sharpe ratio of 1.12 on GLD ($N = 20$, leakage-corrected training protocol), outperforming the buy-and-hold benchmark on a risk-adjusted basis. The system is live, automated, and designed to extend naturally to a multi-asset allocation framework.

Code for this project is available at: https://github.com/Joon-Jeremy-Chun/quant-trading-system

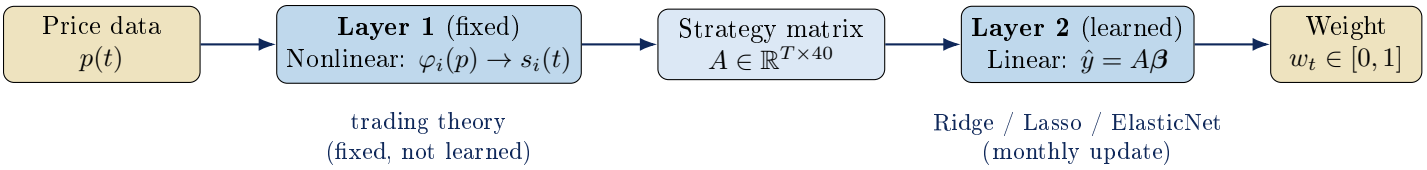
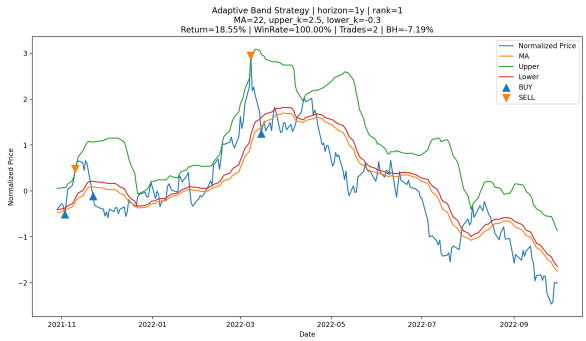
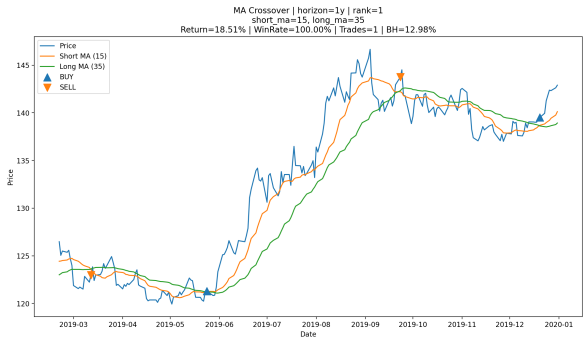


Figure 1: **Two-layer prediction architecture.** Layer 1 converts raw price data into structured strategy signals using fixed, domain-grounded transformations. Layer 2 learns a sparse linear combination of those signals to predict the 130-day forward return, updated monthly on a rolling one-year window.



(a) Adaptive Band Strategy (top-ranked, 1-year window). Signal $+1$ when price exceeds upper band, -1 below lower band.



(b) MA Crossover Strategy (top-ranked, 1-year window). Signal $+1$ when short MA crosses above long MA, -1 when below.

Figure 2: **Example strategy signals from Layer 1.** Each plot shows GLD price (top panel) and the continuous strategy score $s_\theta(t) \in [-1, +1]$ (bottom panel). Different parameter choices θ produce distinct vectors in the strategy space $\mathcal{S}$, spanning different aspects of price dynamics. The two families shown here are among the four used to construct the design matrix A.

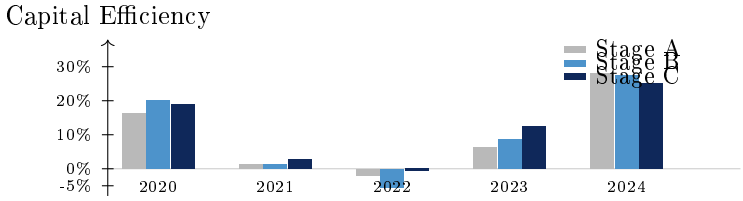


Figure 3: **Capital efficiency by stage and year (GLD, 2020–2024).** Capital efficiency is defined as annual strategy return divided by average capital exposure. Stage C (navy) achieves the highest efficiency in every year. All three stages remain near zero or positive in the 2021 bear year (GLD −6.2%), while Stage C additionally avoids the 2022 drawdown better than Stages A and B.

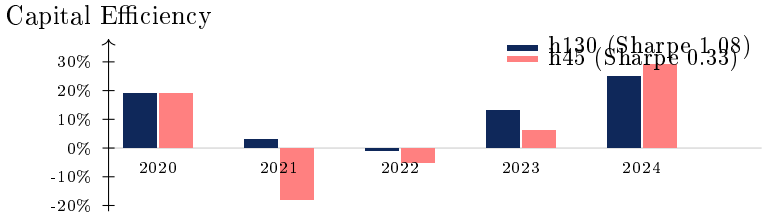


Figure 4: **Horizon comparison: h130 vs h45 capital efficiency (2020–2024, Stage C, top-10).** The h45 model achieves higher peak efficiency in bull years (2024: +29% vs +25%) but fails catastrophically in the 2021 bear year (−18% vs +3%), yielding a Sharpe ratio of 0.33 vs 1.08. The asymmetric failure is attributed to feature–target misalignment: Layer 1 features encode six-month dynamics while h45 targets 45-day returns.

Supplemental Figures

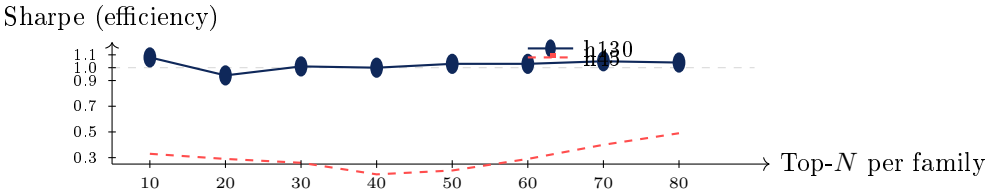


Figure 5: **Supplemental Figure S1: Sharpe ratio vs top-N per family (GLD, 2020–2024).** The h130 Sharpe peaks at $N = 10$ (1.08), drops at $N = 20$ (0.94), and partially recovers to 1.03–1.05 at $N = 50$–70, indicating that the top-10 strategies per family are already near-optimal. The h45 Sharpe remains consistently low across all N (0.17–0.49), confirming that horizon alignment dominates over feature count.

References

Zura Kakushadze. Combining alphas via bounded regression. *Risks*, 3(4):474–490, 2015. doi: 10.3390/risks3040474.

Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1):267–288, 1996. doi: 10.1111/j.2517-6161.1996.tb02080.x.

Reflection

This section is informal and written in first person, as a personal reflection on the research process.

When I started this project, I thought I was building a trading system. By the end, I realized I had accidentally built a framework for thinking about what "features" even mean in a time-series setting—and that turned out to be the more interesting result.

The most surprising thing was the horizon alignment finding. I had assumed that using more data (a longer horizon) would always be safer, or that shorter horizons would at least diversify risk by capturing more cycles. The 2021 bear year completely shattered that intuition. The short-horizon model didn't just underperform—it catastrophically failed in exactly the year where you most need a model to work. When I traced back why, it clicked: the features I was feeding into the regression were calibrated by a six-month evaluation window, and if the regression was trying to predict a 45-day outcome from six-month features, there was a fundamental mismatch baked into the architecture. That connection between the structure of Layer 1 and the appropriate target of Layer 2 was not something I planned; I found it by doing the experiment and asking "why does this fail so badly in 2021?" I think that's how a lot of good research happens.

The most challenging part was keeping honest about what I was actually measuring. Capital efficiency as a metric sounds reasonable, but I spent a lot of time making sure the rolling tranche structure didn't introduce any look-ahead bias—any scenario where today's decision could implicitly use tomorrow's data. The rolling-forward validation design ended up being more thought-intensive than the modeling itself. It's the kind of thing that's easy to get wrong in a way that makes your results look much better than they are, and I kept going back to double-check.

If I could do this again, I would start the multi-asset expansion much earlier. The oracle simulation I ran at the end showed that the theoretical upper bound for a GLD+BRK-B+QQQ system is a Sharpe of 3.02—and I'm currently sitting at 1.08 on GLD alone with real models. That gap between the oracle bound and what the actual model achieves is arguably the most interesting research question in the whole project, and I only got to frame it in the last session before the deadline.

I also learned something about my own research process: I work best when I have a concrete, falsifiable question. The question "does the 130-day horizon outperform the 45-day horizon in a 2021 bear market?" is very answerable. "What is the best trading strategy?" is not. Narrowing the question down to something measurable was consistently the step that unlocked progress. I'll take that forward into any research I do next.